

Modelado con PlantUML: 2 diagramas de casos de uso nivel 0 + 1 diagrama de clases de análisis

Alexander Toapanta -> Coordinador

Julio Blacio -> Modelador PlantUML

Josue Solano -> Analista de requisitos

Kenned Sigcha -> Vocero

1. Desarrollo del reto por rondas

Ronda 1. Radar de actores y objetivos

- Preguntas guía: ¿quién usa la app?, ¿quién administra la información?, ¿qué acciones son visibles para cada actor?
- Evitar errores frecuentes: confundir actor con cargo interno irrelevante, repetir el mismo actor con nombres distintos o convertir pantallas en casos de uso.

Actor	¿Quién es? (Descripción)	¿Quién usa la app? (Objetivo Principal)	¿Quién administra la información? (Acción Clave)	¿Qué acciones son visibles para cada actor? (Ejemplos del SRS)
Administrador	El rol encargado de la gestión operativa y financiera del gimnasio. No es un cargo interno irrelevante, sino un rol con permisos específicos en el sistema.	Gestionar la configuración global del box para su correcto funcionamiento.	Sí. Es el actor que crea y configura la estructura del sistema (clases, membresías, usuarios). También genera reportes.	Crear/editar/eliminar clases (RF-ADM-01). Gestionar tipos de membresía y registrar pagos (RF-ADM-02). Dar de alta, modificar y desactivar cuentas de entrenadores y atletas (RF-ADM-03). Configurar integraciones con apps de salud (CUC-ADM-04). Ver reportes de asistencia y financieros (RF-ADM-04).

Entrenador	El profesional que diseña entrenamientos y realiza el seguimiento de los atletas. Es un usuario clave del sistema, diferenciado del administrador.	Hacer seguimiento del rendimiento de sus atletas y planificar sus rutinas.	Sí, administra la información relacionada con el rendimiento y las rutinas de los atletas que supervisa.	Registrar los resultados de los WODs de un atleta (RF-ENT-04). Crear, modificar y asignar rutinas de entrenamiento personalizadas (RF-ENT-05). Enviar mensajes y anuncios a los atletas (RF-ENT-06). Seguir el rendimiento de atletas en tiempo real (CUC-ENT-01).
Atleta	El cliente del gimnasio que entrena y utiliza la plataforma para gestionar su actividad. Es el usuario final con el mayor volumen.	Reservar clases, registrar su progreso personal y gestionar su cuenta y membresía.	No. Gestiona su propia información personal y de consumo, pero no administra la del sistema ni la de otros usuarios.	Reservar y cancelar clases (RF-ATL-07). Registrar manualmente sus resultados de entrenamiento y ver su progreso (RF-ATL-08). Ver el estado de su membresía, historial de pagos y realizar renovaciones (RF-ATL-09). Recibir mensajes de sus entrenadores.
Sistema de Pagos	Un servicio externo (tercero) que procesa las transacciones financieras. Es un actor no humano.	Procesar los pagos de las membresías de los atletas.	No. Es un sistema externo que ejecuta una tarea específica bajo petición.	Su acción visible es el resultado de una transacción: Pago rechazado (excepción en CUC-ADM-02) o Procesar pago (flujo en CUC-ATL-03).

Apps de Salud	APIs externas (Samsung Health, Google Fit, Apple Health) que proveen datos de salud y actividad física. Es un actor no humano que interactúa con el sistema.	Sincronizar datos biométricos (frecuencia cardíaca, sueño, pasos) automáticamente con la plataforma.	No. Son fuentes de datos externas.	La acción es la sincronización: Sistema importa automáticamente: frecuencia cardíaca, sueño, pasos (flujo alternativo en CUC-ATL-02). Su indisponibilidad es una restricción.
---------------	--	--	------------------------------------	---

Ronda 2 y 3. Casos de uso nivel 0

- Seleccionar casos de uso principales, no subprocesos de bajo nivel.
- Nombrar los casos de uso con verbo en infinitivo: Actualizar perfil, Consultar oportunidades, Registrar retroalimentación, Gestionar noticias, Publicar oportunidades, Generar reportes.
- Usar relaciones <<include>> o <<extend>> solo cuando realmente aporten claridad.

Diagrama de caso de uso del administrador

```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome

actor "Administrador" as Admin
actor "Sistema de Pagos" as Pago
actor "APIs Externas\n(Samsung Health,\nGoogle Fit,\nApple Health)" as API

rectangle "Sistema IRONCLAD BOX" {

    usecase "CUC-ADM-01\nAdministrar la agenda\nde clases" as UC1

    usecase "CUC-ADM-02\nGestionar Membresías\ny Pagos" as UC2

    usecase "CUC-ADM-03\nGestionar Usuarios\ndel Sistema" as UC3

    usecase "CUC-ADM-04\nConfigurar Integraciones\ncon Apps de Salud" as UC4

    ' Subprocesos opcionales para mayor detalle
    usecase "Crear / Editar /\nEliminar clases" as SUB1
    usecase "Asignar entrenador\ny cupos" as SUB2

```

```
usecase "Registrar pagos" as SUB3
usecase "Generar reportes\nfinancieros" as SUB4

usecase "Crear / Modificar /\nDesactivar usuarios" as SUB5
usecase "Asignar roles\ny permisos" as SUB6

usecase "Configurar credenciales\nAPI" as SUB7
usecase "Definir datos a\nsincronizar" as SUB8
}
```

' Relaciones principales

Admin --> UC1

Admin --> UC2

Admin --> UC3

Admin --> UC4

Pago --> UC2

API --> UC4

' Relaciones include

UC1 .> SUB1 : <<include>>

UC1 .> SUB2 : <<include>>

UC2 .> SUB3 : <<include>>

UC2 .> SUB4 : <<include>>

UC3 .> SUB5 : <<include>>

UC3 .> SUB6 : <<include>>

UC4 .> SUB7 : <<include>>

UC4 .> SUB8 : <<include>>

@enduml

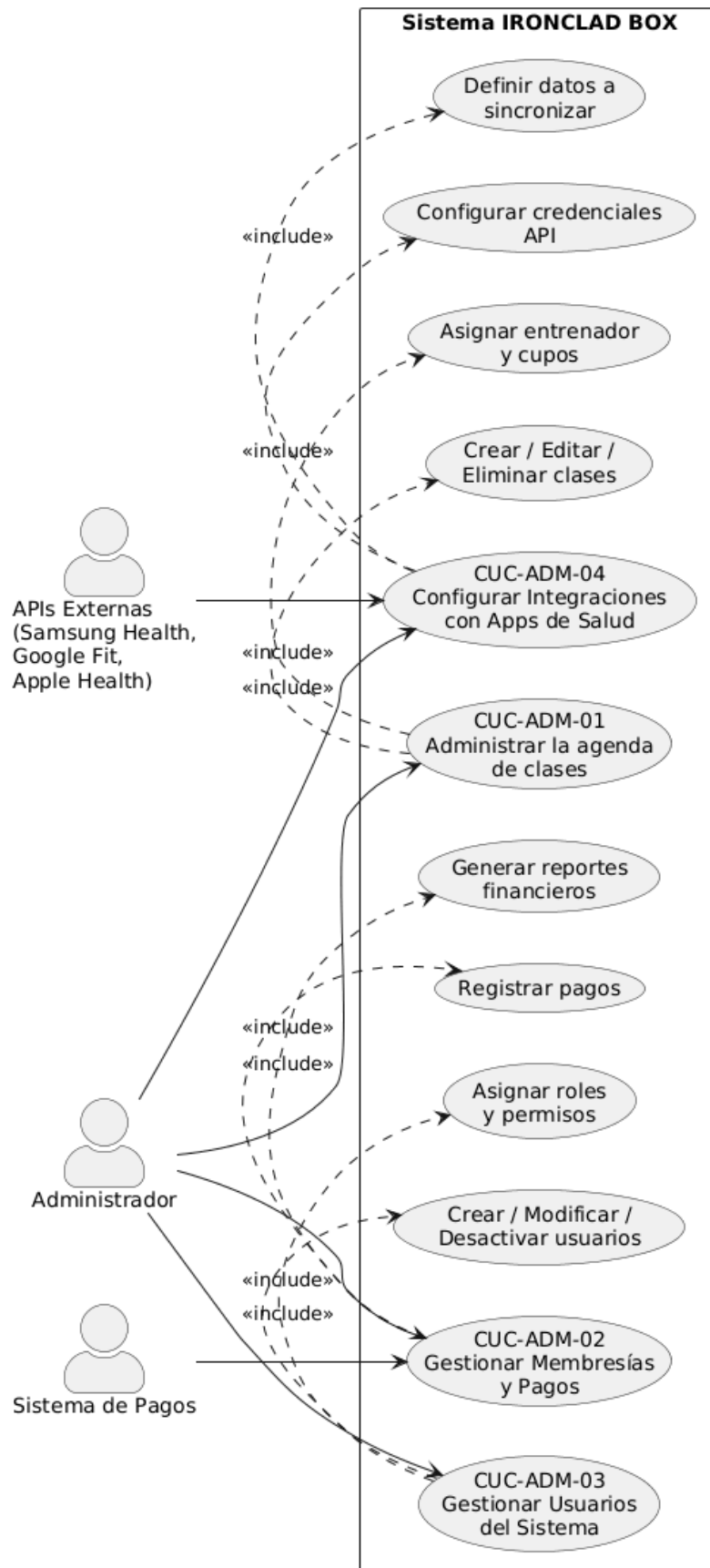


Diagrama de caso de uso del entrenador:

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome
skinparam usecase {
    BorderColor #4A4A4A
    BackgroundColor #F0F0F0
    FontSize 13
}

actor "Entrenador" as Entrenador

rectangle "Sistema IronClad Box" {
    usecase "Autenticarse en el sistema" as Autenticarse
    usecase "Registrar rendimiento\nde atletas" as RegistrarRendimiento
    usecase "Gestionar rutinas\nde entrenamiento" as GestionarRutinas
    usecase "Enviar mensajes\na los atletas" as EnviarMensajes

    usecase "Modificar sesión anterior" as ModificarSesion
    usecase "Crear nueva rutina" as CrearRutina
    usecase "Editar rutina existente" as EditarRutina
}

Entrenador --> Autenticarse
Entrenador --> RegistrarRendimiento
Entrenador --> GestionarRutinas
Entrenador --> EnviarMensajes

RegistrarRendimiento ..> ModificarSesion : <<extend>>
GestionarRutinas ..> CrearRutina : <<extend>>
GestionarsRutinas ..> EditarRutina : <<extend>>

@enduml
```

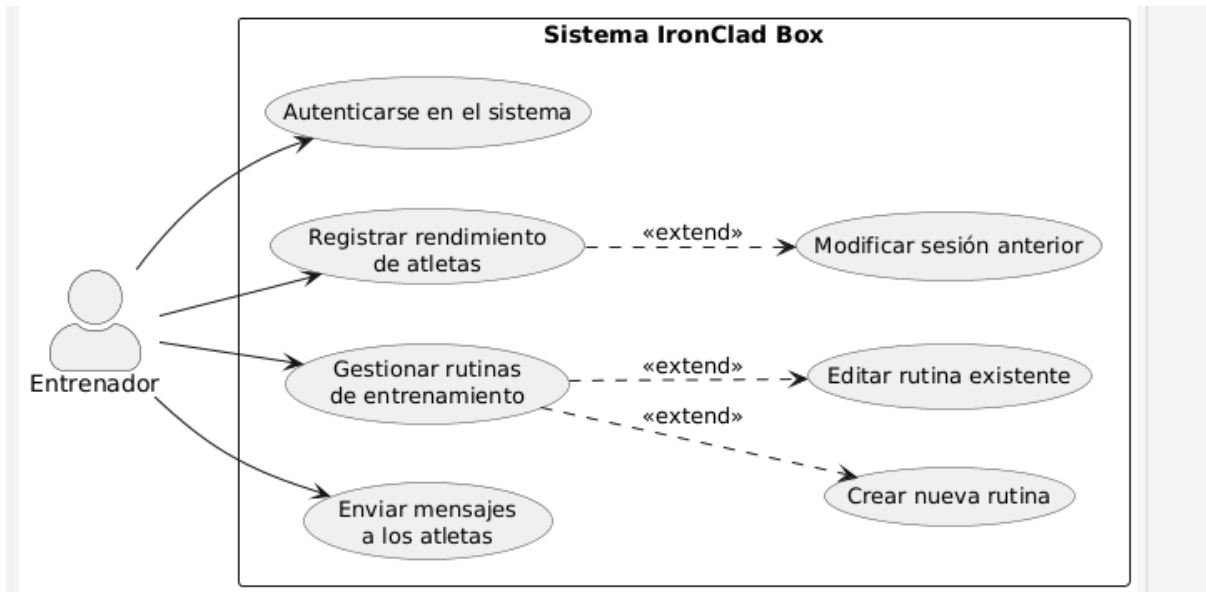


Diagrama de caso de uso del atleta:

```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome
skinparam usecase {
  BorderColor #4A4A4A
  BackgroundColor #F0F0F0
  FontSize 12
}

actor "Atleta" as Atleta
actor "Apps de Salud" as AS <<System>>
actor "Sistema de Pagos" as SP <<System>>

rectangle "IronClad Box - Módulo Atleta" {

  ' Casos de Uso Principales
  usecase "Reservas de Clases" as UC1
  usecase "Registrar\nProgreso de Entrenamiento" as UC2
  usecase "Gestionar\nMembresía Personal" as UC3

  ' Inclusiones UC1
  usecase "Ver horarios disponibles" as UC1_1
  usecase "Reservar y cancelar clases" as UC1_2

  ' Inclusiones UC2
  usecase "Acceder a historial de\nresultados" as UC2_1
  usecase "Visualizar gráficos de progreso" as UC2_2

  ' Inclusiones UC3
  usecase "Ver estado de membresía\ny fecha de vencimiento" as UC3_1
  usecase "Realizar pagos en línea" as UC3_2

  ' Alineación vertical sugerida

```

```

UC1 -[hidden]down- UC2
UC2 -[hidden]down- UC3
}

```

' Relaciones Actor Principal

Atleta --> UC1

Atleta --> UC2

Atleta --> UC3

' Definición de Inclusiones

UC1 ..> UC1_1 : <<include>>

UC1 ..> UC1_2 : <<include>>

UC2 ..> UC2_1 : <<include>>

UC2 ..> UC2_2 : <<include>>

UC3 ..> UC3_1 : <<include>>

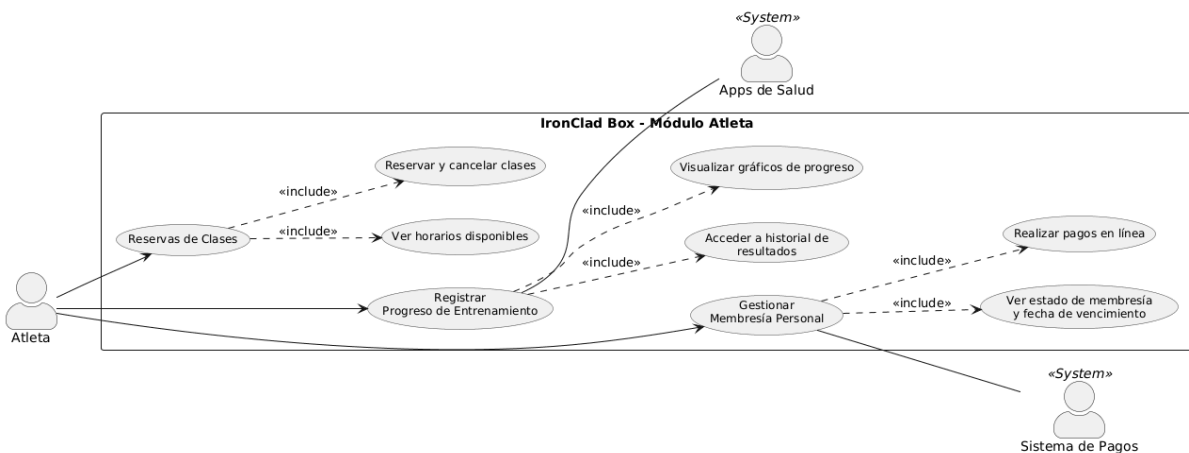
UC3 ..> UC3_2 : <<include>>

' Relaciones con Sistemas Externos (Alineados a la derecha)

UC2 -- AS

UC3 -- SP

@enduml



Ronda 4. Diagrama de clases de análisis

- Extraer clases del dominio a partir de sustantivos relevantes del caso: Egresado, Perfil, Oportunidad, Notificación, Capacitación, Encuesta, Retroalimentación, UsuarioAdministrador.
- No modelar aún detalles de implementación como pantallas, botones, APIs o clases técnicas.
- Priorizar relaciones de asociación, agregación o composición solo si son justificables desde el análisis.


```
@startuml
' Diagrama de Clases de Análisis - IronClad Box
```

```
skinparam class {
    BackgroundColor #FCFCFC
    BorderColor #4A4A4A
    FontSize 12
}
skinparam packageStyle rectangle
```

```
' =====
' Paquete: Usuarios
' =====
```

```
package "Usuarios del Sistema" #F4F4F4 {
```

```
    abstract class Usuario {
        - id : int
        - nombre : string
        - email : string
        - contraseña : string
        - fechaRegistro : date
        --
        + autenticarse() : bool
    }
```

```
    class Administrador {
        - nivelPermisos : string
        --
        + gestionarClases() : void
        + gestionarMembresias() : void
        + gestionarUsuarios() : void
        + generarReportes() : void
    }
```

```
    class Entrenador {
        - especialidad : string
        --
        + registrarRendimiento() : void
        + crearRutina() : void
        + enviarMensaje() : void
    }
```

```
    class Atleta {
        - fechaNacimiento : date
        - telefono : string
        --
        + reservarClase() : void
        + registrarProgreso() : void
        + renovarMembresia() : void
    }
}
```

```
' =====
' Paquete: Núcleo del Negocio
```

```
' =====  
package "Núcleo del Negocio" #F4F4F4 {
```

```
    class Clase {  
        - id : int  
        - dia : date  
        - hora : time  
        - duracion : int  
        - cupoMaximo : int  
        - cuposDisponibles : int  
    }
```

```
    class Membresia {  
        - id : int  
        - tipo : string  
        - precio : float  
        - fechaInicio : date  
        - fechaVencimiento : date  
        - estado : string  
    }
```

```
    class Pago {  
        - id : int  
        - monto : float  
        - fechaPago : date  
        - metodoPago : string  
        - estadoTransaccion : string  
    }
```

```
    class Rutina {  
        - id : int  
        - nombre : string  
        - ejercicios : string  
        - series : int  
        - repeticiones : int  
        - descanso : string  
        - nivelDificultad : string  
    }
```

```
    class RegistroProgreso {  
        - id : int  
        - fecha : date  
        - tiempo : float  
        - repeticiones : int  
        - peso : float  
        - puntuacion : float  
        - notas : string  
    }
```

```
    class Mensaje {  
        - id : int  
        - contenido : string  
        - fechaEnvio : datetime  
        - tipo : string  
    }
```

```

    }

    class Reporte {
        - id : int
        - tipo : string
        - datos : string
        - fechaGeneracion : date
    }
}

' =====
' Paquete: Sistemas Externos
' =====
package "Sistemas Externos" #F4F4F4 {
    class AppSalud {
        - nombre : string
        - apiKey : string
        - ultimaSincronizacion : datetime
        --
        + sincronizarDatos() : object
    }
}

' =====
' Relaciones (Asociaciones)
' =====

' Herencias
Administrador --|> Usuario
Entrenador --|> Usuario
Atleta --|> Usuario

' Asociaciones de Administrador
Administrador "1" -- "0..*" Clase : Gestiona >
Administrador "1" -- "0..*" Membresia : Configura >
Administrador "1" -- "0..*" Reporte : Genera >
Administrador "1" -- "0..*" AppSalud : Configura integración con >

' Asociaciones de Entrenador
Entrenador "1" -- "0..*" Rutina : Crea y Asigna >
Entrenador "1" -- "0..*" RegistroProgreso : Registra >
Entrenador "1" -- "0..*" Mensaje : Envía >

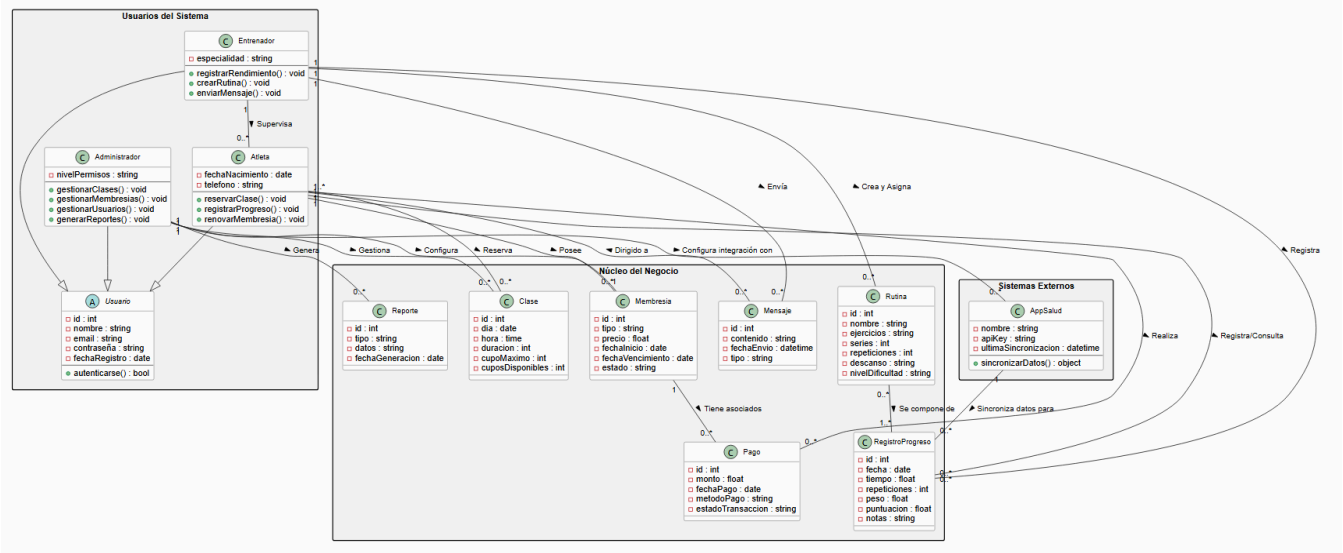
' Asociaciones de Atleta
Atleta "1" -- "1" Membresia : Posee >
Atleta "1" -- "0..*" Pago : Realiza >
Atleta "1" -- "0..*" RegistroProgreso : Registra/Consulta >
Atleta "1..*" -- "0..*" Clase : Reserva >

' Otras Asociaciones
Membresia "1" -- "0..*" Pago : Tiene asociados >
Rutina "0..*" -- "1..*" RegistroProgreso : Se compone de >
Entrenador "1" -- "0..*" Atleta : Supervisa >
Mensaje "0..*" -- "1" Atleta : Dirigido a >

```

AppSalud "1" -- "0..*" RegistroProgreso : Sincroniza datos para >

@enduml



8. Caso base del taller

Caso : Sistema Web "IronClad Box" para la Gestión de Gimnasios de CrossFit.

El gimnasio requiere una plataforma integral que optimice su gestión operativa y mejore la experiencia de sus clientes. El sistema debe permitir la administración centralizada de horarios de clases, cupos y entrenadores asignados. Asimismo, debe gestionar diversos tipos de membresías, el control de estados de pago y la generación de reportes financieros y de asistencia. Para los usuarios finales, la aplicación debe facilitar la reserva y cancelación de clases en línea, el registro manual y automático de resultados de entrenamientos (WODs) mediante la sincronización con APIs de salud (Google Fit, Apple Health) y la visualización de gráficos de progreso personal. Por su parte, los entrenadores deben poder diseñar rutinas personalizadas y mantener una comunicación directa con los atletas para brindar feedback y anuncios.

9. Entregable del equipo

- Lista de actores identificados: Administrador, Entrenador, Atleta, Sistema de Pagos y Apps de Salud.
- Código PlantUML del diagrama de casos de uso: Módulo Atleta. (Enfocado en CUC-ATL-01, 02 y 03) .
- Código PlantUML del diagrama de casos de uso: Módulo Administrador. (Enfocado en CUC-ADM-01 al 04) .

- Código PlantUML del diagrama de clases de análisis. (Modelado del dominio: Usuario, Clase, Membresía, Pago, Rutina, etc.) .
- Justificación de trazabilidad: Los modelos presentados derivan directamente de los requisitos funcionales (RF) y casos de uso del sistema (CUC) definidos en la especificación de requisitos de IronClad Box.